

INFORME TECNICO DE PROYECTO

Samuel Calderón Candela

SOC Analyst / Blue Team

Laboratorio SOC / Blue Team — Detección, Analisis y Respuesta

Caso de estudio: Incidente 001

Entorno: Wazuh 4.12.0 + Sysmon sobre endpoint Windows

Fecha del incidente analizado: 25 de agosto de 2026 (UTC)

Clasificación del documento: Portafolio tecnico — SOC Analyst / Junior Blue Team

1. Introducción

Este documento presenta el informe técnico del laboratorio SOC / Blue Team desarrollado de forma casera, cuyo objetivo fue construir un entorno funcional de monitoreo, detección y análisis de seguridad utilizando Wazuh como SIEM y Sysmon como fuente de telemetría en un endpoint Windows. El proyecto reproduce, a escala reducida, el flujo de trabajo real de un analista SOC de Nivel 1: generación de telemetría, detección, triage, análisis, correlación, reconstrucción de línea de tiempo, evaluación de impacto y documentación.

El núcleo analítico del informe es el Incidente 001, una simulación controlada y autorizada de una cadena de ejecución sospechosa (Notepad -> CMD -> PowerShell -> PowerShell codificado -> conexión de red saliente). El resto de las secciones documentan el contexto técnico necesario para comprender dicho incidente.

2. Objetivo del Proyecto

El objetivo general del proyecto fue construir y operar un entorno práctico de laboratorio SOC utilizando Wazuh y un endpoint Windows instrumentado con Sysmon, demostrando el ciclo completo del analista SOC:

- Generar telemetría
- Detectar
- Realizar triage
- Analizar
- Correlacionar
- Reconstruir
- Evaluar
- Responder
- Documentar

El proyecto busca demostrar capacidades relevantes para los perfiles de SOC Analyst Tier 1, Junior Blue Team Analyst, Security Operations, Incident Response y Detection Engineering, con foco en monitoreo de endpoints Windows.

El objetivo NO fue convertirse en administrador de Wazuh ni construir un SIEM de nivel empresarial. El objetivo fue demostrar la capacidad de interpretar telemetría real, investigar alertas, correlacionar comportamiento de procesos, reconstruir una cadena de ataque, identificar actividad sospechosa, mapearla a MITRE ATT&CK, realizar triage y documentar una evaluación de seguridad defendible.

La metodología del proyecto prioriza la investigación práctica y la telemetría real por sobre el aprendizaje puramente teórico.

3. Alcance

El alcance del proyecto comprende el despliegue de un servidor Wazuh, la instrumentación de un endpoint Windows con Sysmon y el Agente Wazuh, la configuración de reglas de detección locales, la generación de una línea base de actividad legítima y la ejecución de una simulación de ataque controlada (Incidente 001) utilizando Kali Linux como plataforma ofensiva.

Quedan fuera del alcance la administración avanzada de Wazuh/OpenSearch, la construcción de una arquitectura SIEM de producción, y la ejecución de técnicas ofensivas adicionales mas allá de las necesarias para validar el flujo de detección definido.

4. Arquitectura del Laboratorio

El laboratorio se compone de tres elementos principales: el servidor SIEM (Wazuh), el endpoint Windows monitoreado y la plataforma de simulación ofensiva (Kali Linux).

4.1 Diagrama logico

```
Kali Linux (192.168.1.21) --TCP 4444--> WIN10-SOC-ENDPOINT01 WIN10-SOC-ENDPOINT01
(Sysmon + Wazuh Agent) --telemetría--> Wazuh Manager / Indexer / Dashboard (Ubuntu
Server)
```

5. Configuración del Entorno

5.1 SIEM / Servidor

- Ubuntu Server 24.04.4
- Wazuh Manager, Wazuh Indexer/OpenSearch y Wazuh Dashboard 4.12.0
- Aproximadamente 3.5 GiB de RAM, 2 núcleos de CPU, swap configurado

5.2 Endpoint Windows

- Hostname: WIN10-SOC-ENDPOINT01
- Agente Wazuh instalado
- Sysmon instalado y configurado como fuente principal de telemetría
- Activo principal monitoreado durante el proyecto

5.3 Plataforma de simulación ofensiva

Kali Linux fue utilizado exclusivamente como plataforma controlada de ataque/simulación dentro del laboratorio. Durante el Incidente 001, Kali opero desde la dirección 192.168.1.21, con un listener TCP controlado en el puerto 4444.

6. Fuentes de Telemetría

La fuente principal de telemetría utilizada en la investigación es Sysmon Event ID 1 (creación de procesos), la cual fue ingerida exitosamente de forma end-to-end en Wazuh. Los campos disponibles y

utilizados incluyen Image, ParentImage, CommandLine, ParentCommandLine, User, ParentUser, IntegrityLevel, ProcessId, ParentProcessId, ProcessGuid, ParentProcessGuid, SHA256 y marcas de tiempo (UTC).

ProcessGuid y ParentProcessGuid son especialmente relevantes para este proyecto: a diferencia de ProcessId/ParentProcessId, que el sistema operativo puede reutilizar en el tiempo, los GUID son identificadores únicos generados por Sysmon para cada proceso, lo que los convierte en la única base confiable para reconstruir relaciones padre/hijo entre procesos a lo largo de una investigación.

Sysmon Event ID 3 (conexión de red) fue habilitado y confirmado a nivel de Windows, pero no fue ingerido exitosamente por Wazuh 4.12.0 en este entorno. Esta limitación se documenta en la sección 11 y se explica como fue compensada durante la investigación del Incidente 001 mediante validación directa en el endpoint y en el host remoto controlado.

7. Ingeniería de Detección

La ingeniería de detección del laboratorio se centro en la creación de reglas locales de Wazuh sobre eventos Sysmon EID 1, orientadas a detectar relaciones padre/hijo de procesos consideradas de interés para un analista SOC (por ejemplo, interpretes de comandos generados por aplicaciones no administrativas, o uso de parámetros de ofuscación en PowerShell). El diseño de reglas prioriza la detección de comportamiento (relaciones de procesos, parámetros de línea de comandos) por sobre la detección basada únicamente en nombres de archivo.

8. Reglas de Wazuh Configuradas

Se configuraron las siguientes reglas locales:

Regla	Nombre	Proposito
100100	SOC Lab - Notepad process creation	Detectar la creación del proceso notepad.exe.
100102	SOC Lab - PowerShell executed with encoded command	Detectar ejecución de PowerShell con el parametro - EncodedCommand.
100104	SOC Lab - PowerShell spawned by CMD	Detectar PowerShell cuando es generado por cmd.exe.
100105	SOC Lab - CMD spawned by Notepad	Detectar cmd.exe cuando es generado por notepad.exe.
100106	SOC Lab - tuning exception	Reducir un falso positivo legitimo generado por actividad del Agente Wazuh relacionada con net.exe.

La validación funcional de las reglas 100102, 100104 y 100105 se demuestra únicamente a través de la evidencia del Incidente 001 (secciones 10.5 a 10.8), y no se considera confirmada por su sola existencia en la configuración.

9. Línea Base de Actividad Legítima

Previo a la ejecución del Incidente 001, se observó una ejecución legítima de Notepad, generada por el usuario a través del Explorador de Windows. La relación de procesos observada fue explorer.exe -> notepad.exe, lo cual disparó la regla 100100.

Imagen	C:\Windows\System32\notepad.exe
Proceso padre	C:\Windows\explorer.exe
CommandLine	"C:\WINDOWS\system32\notepad.exe"
ParentCommandLine	C:\WINDOWS\Explorer.EXE
Usuario / Usuario padre	DESKTOP-H9IT5UI\WIN-CLIENT01 (ambos)
IntegrityLevel	Medium
Compania (firma)	Microsoft Corporation
SHA256	DA5807BB0997CC6B5132950EC87EDA2B33B1AC4533CF1F7A22A6F3B576ED7C5B
UTC	2026-08-24 23:54:23.158
Regla disparada	100100 — SOC Lab - Notepad process creation

alerta legitima de Notepad (Regla 100100)]

Aug 24, 2026 @ 19:01:29.041 input.type: log agent.ip: 192.168.1.7 agent.name: WIN10-SOC-ENDPOINT01 agent.id: 001 manager.name: SRV-SIEM01
data.win.eventdata.originalFileName: NOTEPAD.EXE data.win.eventdata.image: C:\Windows\System32\notepad.exe
data.win.eventdata.product: Microsoft Windows Operating System data.win.eventdata.parentProcessGuid: {812dfa9a-60a1-6a8c-6004-00000001600}
data.win.eventdata.description: Notepad data.win.eventdata.logonGuid: {812dfa9a-6094-6a8c-14c2-8c0100000000}
data.win.eventdata.parentCommandLine: C:\WINDOWS\Explorer.EXE data.win.eventdata.processGuid: {812dfa9a-db58-6a8c-b906-0000000016}

Expanded document View surrounding documents View single document

[ANALISIS] El evento cumple con todos los indicadores de ejecución legítima: ruta estandar del sistema, firma de Microsoft Corporation, proceso padre esperado (explorer.exe), mismo usuario en proceso padre e hijo, IntegrityLevel Medium, y ausencia de procesos hijo sospechosos en este evento.

El punto analítico central de esta sección es que una alerta de Wazuh no equivale automaticamente a un incidente de seguridad. La regla 100100 detecto correctamente la creación del proceso Notepad, pero el análisis de contexto (ruta, firma, proceso padre, usuario, integridad) permitio clasificar la actividad como Benigna / Esperada, estableciendo asi la línea base contra la cual se compara el comportamiento observado en el Incidente 001.

10. Incidente 001

Incidente 001 constituye el caso de estudio principal del proyecto y el componente de mayor profundidad analítica del informe.

10.1 Escenario del Incidente

La cadena de ataque conceptual planificada para la simulación fue: aplicación legítima (Notepad) -> CMD -> PowerShell -> comando codificado -> actividad de red controlada hacia Kali Linux. Es importante remarcar que esta secuencia representa el escenario previsto para la simulación, y no debe asumirse

automáticamente como la cadena final confirmada; la reconstrucción real se realiza a partir de la telemetría efectivamente recolectada (secciones 10.9 y 10.10).

10.2 Simulación del Ataque

La simulación fue ejecutada de forma autorizada y controlada dentro del laboratorio, generando en el endpoint Windows la secuencia de procesos Notepad -> CMD -> PowerShell -> PowerShell con - EncodedCommand, culminando en el establecimiento de una conexión TCP saliente hacia el listener controlado en Kali Linux (192.168.1.21:4444).

10.3 Detección

La totalidad de la cadena fue detectada por las reglas locales de Wazuh 100105 (CMD generado por Notepad), 100104 (PowerShell generado por CMD) y 100102 (PowerShell con parametro - EncodedCommand), sobre eventos Sysmon EID 1 ingeridos correctamente en el Wazuh Manager.

10.4 Triage Inicial

Ante la aparición de las tres alertas en una ventana temporal cercana y sobre el mismo host y usuario, el triage inicial elevó la prioridad de revisión debido a la combinación de indicadores: aplicación no administrativa generando un interprete de comandos, interprete de comandos generando PowerShell, y uso de un parámetro de ofuscación (-EncodedCommand). Esta combinación es consistente con técnicas de ejecución y evasión documentadas en MITRE ATT&CK, lo que justificó proceder con un análisis detallado en lugar de un cierre automático.

10.5 Evidencia Confirmada — Etapa 1: Notepad -> CMD

Imagen	C:\Windows\System32\cmd.exe
Proceso padre (ParentImage)	C:\Windows\System32\notepad.exe
CommandLine	C:\Windows\System32\cmd.exe
ParentCommandLine	"C:\WINDOWS\system32\notepad.exe"
ProcessId / ParentProcessId	7852 / 3616
ProcessGuid	{812dfa9a-043b-6a8d-d807-000000001600}
ParentProcessGuid	{812dfa9a-fb42-6a8c-9907-000000001600}
UTC	2026-08-25 02:55:55.549
Usuario / Usuario padre	DESKTOP-H9IT5UI\WIN-CLIENT01 (ambos)
IntegrityLevel	Medium
SHA256	BADF4752413CB0CBDC03FB95820CA167F0CDC63B597CCDB5EF43111180E088B0
Regla disparada	100105 — SOC Lab - CMD spawned by Notepad

Notepad -> CMD, Regla 100105 y relación padre/hijo

```
Aug 24, 2026 @ 21:55:56.675 input.type: log agent.ip: 192.168.1.7 agent.name: WIN10-SOC-ENDPOINT01 agent.id: 001 manager.name: SRV-SIEM01
data.win.eventdata.originalFileName: Cmd.Exe data.win.eventdata.image: C:\Windows\System32\cmd.exe
data.win.eventdata.product: Microsoft Windows Operating System data.win.eventdata.parentProcessGuid: {812dfa9a-fb42-6a8c-9907-000000001600}
data.win.eventdata.description: Windows Command Processor data.win.eventdata.logonGuid: {812dfa9a-6094-6a8c-14c2-8c0100000000}
data.win.eventdata.parentCommandLine: "C:\WINDOWS\system32\notepad.exe" data.win.eventdata.processGuid: {812dfa9a-043b-
```

[EVIDENCIA CONFIRMADA] Un proceso notepad.exe (PID 3616) genero un proceso cmd.exe (PID 7852) bajo el mismo usuario, con IntegrityLevel Medium, en el mismo endpoint. La relación queda confirmada por ParentProcessGuid={812dfa9a-fb42-6a8c-9907-000000001600} coincidiendo con el ProcessGuid del Notepad de la línea base temporalmente mas cercano.

Esta relación padre/hijo (Notepad generando un interprete de comandos) es atípica en un flujo de uso normal de un editor de texto, y por lo tanto constituye un comportamiento de proceso sospechoso según cualquier criterio de detección basado en comportamiento.

La regla 100105 detecto correctamente la relación que fue diseñada para identificar. La actividad en si misma fue generada de forma intencional como parte de la simulación autorizada de laboratorio. Es fundamental distinguir tres conceptos que suelen confundirse en el triage:

- Detección correcta: la regla identifico exactamente el comportamiento para el que fue diseñada (Notepad generando CMD).
- Comportamiento sospechoso: la relación de procesos observada es anómala y justificaría investigación en un entorno de producción.
- Contexto autorizado: en este laboratorio, la actividad fue generada deliberadamente y con autorización, por lo que no representa una amenaza real.

[EVALUACION] La clasificación final de esta etapa es Benigna / Validación de Seguridad Autorizada. No debe describirse la regla 100105 como un falso positivo, ya que un falso positivo implica que la regla disparo sin que el comportamiento detectado haya ocurrido realmente; aquí el comportamiento si ocurrió tal como fue detectado.

10.6 Evidencia Confirmada — Etapa 2: CMD -> PowerShell

Imagen	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Proceso padre (ParentImage)	C:\Windows\System32\cmd.exe
ParentCommandLine	"C:\WINDOWS\system32\cmd.exe"
ProcessId / ParentProcessId	7036 / 9272
ProcessGuid	{812dfa9a-0445-6a8d-da07-000000001600}
ParentProcessGuid	{812dfa9a-0365-6a8d-d407-000000001600}
UTC	2026-08-25 02:56:05.754
Usuario / Usuario padre	DESKTOP-H9IT5UI\WIN-CLIENT01 (ambos)
IntegrityLevel	Medium
SHA256	9785001B0DCF755EDDB8AF294A373C0B87B2498660F724E76C4D53F9C217C7A3
Regla disparada	100104 — SOC Lab - PowerShell spawned by CMD

[CMD -> PowerShell, Regla 100104]

```
Aug 24, 2026 @ 22:03:21.089 input.type: log agent.ip: 192.168.1.7 agent.name: WIN10-SOC-ENDPOINT01 agent.id: 001 manager.name: SRV-SIEM01
data.win.eventdata.originalFileName: PowerShell.EXE data.win.eventdata.image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
data.win.eventdata.product: Microsoft Windows Operating System data.win.eventdata.parentProcessGuid: {812dfa9a-dddb-6a8c-d606-000000001600}
data.win.eventdata.description: Windows PowerShell data.win.eventdata.logonGuid: {812dfa9a-6094-6a8c-14c2-8c0100000000}
data.win.eventdata.parentCommandLine: \"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe\"
```

[EVIDENCIA CONFIRMADA] Un proceso powershell.exe (PID 7036, ProcessGuid {812dfa9a-0445-6a8d-da07-000000001600}) fue generado por un proceso cmd.exe (ParentProcessGuid {812dfa9a-0365-6a8d-d407-000000001600}) 10.2 segundos después del evento de la Etapa 1.

Nota de correlación: el ParentProcessGuid de este evento ({812dfa9a-0365-6a8d-d407-000000001600}) no coincide con el ProcessGuid del cmd.exe documentado en la Etapa 1 ({812dfa9a-043b-6a8d-d807-000000001600}). Esto indica que el proceso cmd.exe que genero este PowerShell es una instancia de cmd.exe distinta a la registrada en la Etapa 1, y no puede confirmarse mediante GUID que ambos eventos correspondan al mismo proceso cmd.exe. Esta observación se retoma con mayor profundidad en la sección 10.9.

La relevancia de seguridad de este evento no radica en la presencia de PowerShell por si sola (un binario legítimo y de uso extremadamente común en Windows), sino en su relación de proceso padre: PowerShell generado directamente por un interprete de comandos (cmd.exe) que, a su vez, no proviene de un flujo administrativo esperado. Esta cadena de interpretes anidados (Notepad-like -> CMD -> PowerShell) es un patrón mas asociado a ejecución de comandos por script o por un operador que a uso interactivo normal de PowerShell desde una terminal abierta directamente por el usuario.

10.7 Evidencia Confirmada — PowerShell Codificado

Imagen	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Proceso padre (ParentImage)	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
ParentCommandLine	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
ProcessId / ParentProcessId	10092 / 8124
ProcessGuid	{812dfa9a-05f7-6a8d-f207-000000001600}
ParentProcessGuid	{812dfa9a-dddb-6a8d-d606-000000001600}
UTC	2026-08-25 03:03:19.272
Usuario / Usuario padre	DESKTOP-H9IT5UI\WIN-CLIENT01 (ambos)
IntegrityLevel	Medium
SHA256	9785001B0DCF755EDDB8AF294A373C0B87B2498660F724E76C4D53F9C217C7A3
Regla disparada	100102 — SOC Lab - PowerShell executed with encoded command

PowerShell con -EncodedCommand, Regla 100102]

```
Aug 24, 2026 @ 21:56:07.905 input.type: log agent.ip: 192.168.1.7 agent.name: WIN10-SOC-ENDPOINT01 agent.id: 001 manager.name: SRV-SIEM01
data.win.eventdata.originalFileName: PowerShell.EXE data.win.eventdata.image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
data.win.eventdata.product: Microsoft Windows Operating System data.win.eventdata.parentProcessGuid: {812dfa9a-0365-6a8d-d407-000000001600}
data.win.eventdata.description: Windows PowerShell data.win.eventdata.logonGuid: {812dfa9a-6094-6a8c-14c2-8c0100000000}
data.win.eventdata.parentCommandLine: \"C:\WINDOWS\system32\cmd.exe\" data.win.eventdata.processGuid: {812dfa9a-0445-6
```


10.7.1 CommandLine observado

```
"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -EncodedCommand JABjAD0ATgB1AHcALQBPAGIAagB1AGMAdAAGAFMAeQBzAHQAZQBtAC4ATgB1AHQALgBTAG8AY2tldHMuVABjcENSaWVudDskYy5Db25uZWNOKCCcxOTIuMTY4LjEuMjEnLDQ0NDQpO1N0YXJ0LVNsZWVwIC1TZWNvbmRzIDYwOyRjLkNsb3NlKCK=
```

10.7.2 Decodificación del comando (Base64 / UTF-16LE)

```
$c=New-Object System.Net.Sockets.TcpClient;$c.Connect('192.168.1.21',4444);Start-Sleep -Seconds 60;$c.Close()
```

Análisis funcional del comando decodificado:

- Crea un objeto System.Net.Sockets.TcpClient (clase nativa de .NET para conexiones TCP salientes).
- Establece conexión hacia la dirección IP 192.168.1.21 en el puerto TCP 4444.
- Mantiene la conexión abierta durante aproximadamente 60 segundos (Start-Sleep).
- Cierra la conexión (Close).

[ANALISIS] El comando decodificado no contiene lógica de lectura/escritura de flujos (NetworkStream), ejecución remota de comandos, ni redirección de entrada/salida hacia la conexión. Por lo tanto, técnicamente el comando establece y mantiene una conexión TCP saliente, pero no implementa un shell interactivo ni transferencia de comandos/datos observable en el propio script.

En consecuencia, la descripción técnicamente correcta de esta actividad es una simulación de comunicación tipo C2 mediante una conexión TCP saliente controlada, y no una shell inversa interactiva, ni evidencia de ejecución remota de comandos, ni una compromisión real de C2. El uso de -EncodedCommand por si mismo constituye una tecnica de ofuscación reconocida (el analista no puede leer la intención del comando sin decodificarlo previamente), independientemente de la simplicidad del payload subyacente.

10.8 Reconstrucción de la Línea de Tiempo

UTC	Evento	Detección
23:54:23.158 (24/08)	Ejecucion legitima de Notepad (línea base)	100100
02:55:55.549	notepad.exe genera cmd.exe (PID 7852)	100105
02:56:05.754	cmd.exe genera powershell.exe (PID 7036); +10.2 s	100104
03:03:19.272	powershell.exe genera powershell.exe con -EncodedCommand (PID 10092); +7 min 13.5 s desde el evento anterior	100102
~03:03:19 +	Conexion TCP saliente hacia 192.168.1.21:4444, sostenida ~60 s	Validación manual (netstat / Netcat) — ver 10.10

[ANALISIS] Entre el evento de las 02:56:05.754 (CMD->PowerShell) y el evento de las 03:03:19.272 (PowerShell codificado) transcurren aproximadamente 7 minutos y 13 segundos, un intervalo significativamente mayor al observado entre los eventos anteriores (10.2 segundos). Esta diferencia temporal es consistente con -y refuerza- el hallazgo de correlación de la sección 10.9: no hay indicios de que ambos eventos formen parte de una única ejecución continua e ininterrumpida.

10.9 Hallazgo Crítico de Correlación de Procesos

Este es uno de los puntos analíticos mas importantes del Incidente 001.

El escenario conceptual previsto para la simulación (sección 10.1) sugiere una única cadena continua: Notepad -> CMD -> PowerShell -> PowerShell codificado, como si cada proceso fuera hijo directo del anterior en una sola secuencia de ejecución. La evidencia de telemetría, sin embargo, no permite confirmar esa cadena continua cuando se examina a nivel de ProcessGuid/ParentProcessGuid.

10.9.1 Lo que la telemetría confirma

- notepad.exe (ProcessGuid {812dfa9a-fb42-6a8c-9907-000000001600}) genero cmd.exe (ProcessGuid {812dfa9a-043b-6a8d-d807-000000001600}). Relacion confirmada por GUID.
- Un proceso cmd.exe (ProcessGuid {812dfa9a-0365-6a8d-d407-000000001600}) genero el primer powershell.exe (ProcessGuid {812dfa9a-0445-6a8d-da07-000000001600}). Relación confirmada por GUID, pero este ProcessGuid de cmd.exe no coincide con el ProcessGuid del cmd.exe documentado en la Etapa 1.
- Un proceso powershell.exe (ProcessGuid {812dfa9a-dddb-6a8d-d606-000000001600}) genero el powershell.exe con -EncodedCommand (ProcessGuid {812dfa9a-05f7-6a8d-f207-000000001600}). Relacion confirmada por GUID, pero este ProcessGuid del PowerShell padre no coincide con el ProcessGuid del primer powershell.exe documentado en la Etapa 2 ({812dfa9a-0445-6a8d-da07-000000001600}).

10.9.2 Lo que se esperaba originalmente

El diseño conceptual de la simulación (sección 10.1) suponía que el mismo proceso cmd.exe de la Etapa 1 seria el que generara el PowerShell de la Etapa 2, y que ese mismo PowerShell seria el que ejecutara finalmente el comando codificado, formando una única rama continua en el árbol de procesos.

10.9.3 Lo que la telemetría realmente demuestra

La telemetría demuestra tres relaciones padre/hijo validas e independientes (Notepad->CMD confirmada por GUID; CMD->PowerShell confirmada por GUID; PowerShell->PowerShell-codificado confirmada por GUID), pero no demuestra que estas tres relaciones formen una única cadena ininterrumpida de descendencia directa entre el primer y el ultimo evento. Los ProcessGuid de los procesos intermedios (cmd.exe y el primer powershell.exe) referenciados como "padre" en cada salto no coinciden con los ProcessGuid de los procesos "hijo" documentados en el salto anterior.

10.9.4 Lo que no puede confirmarse

[INFERENCIA] Con la telemetría disponible no puede confirmarse que el proceso powershell.exe que ejecuto el comando codificado sea descendiente directo, dentro de una misma rama continua del árbol

de procesos, del cmd.exe generado por Notepad en la Etapa 1. Es posible que hayan existido procesos intermedios adicionales no capturados en la evidencia suministrada, o que se hayan abierto múltiples instancias independientes de cmd.exe/powershell.exe durante la ventana de la simulación. Ninguna de estas posibilidades puede confirmarse ni descartarse con los datos disponibles.

10.9.5 Por que este hallazgo es relevante

Este hallazgo no debe ocultarse ni forzarse para que la evidencia encaje con la hipótesis original del escenario. Por el contrario, constituye una demostración práctica y central del proyecto: ilustra por que ProcessId y ParentProcessId por si solos son insuficientes para una reconstrucción confiable (al ser reutilizables por el sistema operativo), y por qué el analista debe validar cada eslabón de una cadena de procesos mediante ProcessGuid/ParentProcessGuid antes de afirmar continuidad. Reportar una cadena continua sin esa validación habría constituido una inferencia presentada como evidencia confirmada, lo cual esta explícitamente fuera de los estándares de rigor de este informe.

[CONCLUSION] Cada salto individual de la cadena (Notepad->CMD, CMD->PowerShell, PowerShell->PowerShell codificado) está confirmado por su propio par ProcessGuid/ParentProcessGuid. Lo que no está confirmado es la continuidad de una única rama ininterrumpida entre el primer y el ultimo evento de la secuencia.

10.10 Validación de Red

Durante la ejecución del PowerShell codificado, el endpoint estableció una conexión TCP hacia 192.168.1.21:4444, donde Kali Linux mantenía un listener activo mediante Netcat.

Dado que Sysmon Event ID 3 no fue ingerido exitosamente por Wazuh en este entorno (sección 6 y sección 11), la conexión de red no pudo validarse a través de una alerta de Wazuh. Para compensar esta limitación de telemetría, la validación se realizó de forma independiente en ambos extremos de la comunicación:

- En el endpoint Windows, mediante el comando netstat -ano, observando la conexión activa hacia 192.168.1.21:4444 junto con su PID asociado.
- En Kali Linux, mediante la observación directa de la conexión entrante en el listener de Netcat.

— salida de netstat -ano en Windows mostrando la conexión activa hacia 192.168.1.21:4444 y PID :

```
C:\Users\WIN-CLIENT01>netstat -ano | findstr :4444
TCP        192.168.1.7:51926      192.168.1.21:4444      ESTABLISHED    7036
```

[EVIDENCIA CONFIRMADA] Wazuh no detecto esta conexión mediante Sysmon EID 3 (no ingerido). La validación de la actividad de red se realizó mediante verificación manual, directa y correlacionada en el endpoint (netstat) y en el host remoto controlado (Netcat), no mediante una alerta generada por el SIEM.

10.11 Correlación

La correlación del incidente combina cuatro fuentes de evidencia: las tres alertas de Wazuh (100105, 100104, 100102) generadas sobre eventos Sysmon EID 1 en el mismo host y usuario (DESKTOP-H9IT5UI\WIN-CLIENT01) dentro de una ventana de aproximadamente 7.5 minutos, y la validación de red

independiente (netstat en Windows + Netcat en Kali) coincidente en tiempo con el ultimo evento de PowerShell codificado.

Esta correlación multi-fuente permite reconstruir un cuadro coherente del comportamiento observado en el endpoint, incluso considerando que la continuidad estricta de la rama del árbol de procesos no pudo confirmarse en su totalidad (sección 10.9) y que la telemetría de red nativa de Wazuh no estuvo disponible (sección 6).

10.12 Mapeo MITRE ATT&CK

Tecnica	Nombre	Justificacion (evidencia observada)
T1059.003	Windows Command Shell	Ejecucion de cmd.exe generado por notepad.exe (Etapa 1, Regla 100105).
T1059.001	PowerShell	Ejecucion de powershell.exe generado por cmd.exe y, posteriormente, por otro proceso powershell.exe (Etapas 2 y 3, Reglas 100104 y 100102).
T1027	Obfuscated/Compressed Files or Information	Uso del parametro -EncodedCommand con payload en Base64/UTF-16LE, que oculta el contenido del comando hasta su decodificacion (Regla 100102).

[EVALUACION] La conexión TCP saliente hacia 192.168.1.21:4444 no se asigna a una técnica específica de Comando y Control (por ejemplo, T1071 - Application Layer Protocol, o T1571 - Non-Standard Port). El comando decodificado únicamente abre, mantiene y cierra un socket TCP, sin protocolo de aplicación identificable, sin trafico de comandos observado y sin persistencia. Asignar una técnica de C2 específica a partir de una única conexión TCP simple constituiría una sobre-interpretacion no respaldada por la evidencia disponible.

10.13 Evaluación de Impacto

El impacto confirmado por la evidencia se limita a lo siguiente:

- Ejecución de procesos controlada dentro del endpoint de laboratorio (Notepad, CMD, PowerShell).
- Ejecución de un comando PowerShell codificado (ofuscación T1027).
- Establecimiento de una conexión TCP saliente controlada hacia un host de laboratorio autorizado.
- Generación y detección exitosa de telemetría a través de las reglas locales de Wazuh.

[CONCLUSION] No se observa evidencia de persistencia, escalada de privilegios, movimiento lateral, exfiltración de datos, actividad de ransomware, C2 real ni compromiso del sistema. Ninguno de estos impactos fue demostrado por la telemetría disponible y, por lo tanto, no se afirman en este informe.

10.14 Clasificación del Incidente

10.14.1 Perspectiva de producción

Si esta misma combinación de comportamientos (Notepad generando CMD, CMD generando PowerShell, PowerShell ejecutando un comando codificado, conexión saliente hacia un host externo por un puerto no estándar, sobre el mismo usuario y host) apareciera de forma inesperada en un endpoint de producción, justificaría una investigación de prioridad alta, dada la combinación de indicadores de ejecución (T1059) y ofuscación (T1027).

10.14.2 Perspectiva de laboratorio

En este proyecto, la actividad fue generada de forma deliberada y autorizada como parte de la simulación. Por lo tanto, la clasificación final del incidente es Benigno / Validación de Seguridad Autorizada, con disposición Cerrado — Simulación de Incidente Controlada.

[EVALUACION] Esta clasificación no equivale a calificar las detecciones como falsos positivos. Las reglas 100105, 100104 y 100102 detectaron correctamente los comportamientos para los cuales fueron diseñadas; lo que determina la clasificación final benigna es el contexto de autorización del laboratorio, no una falla de detección.

10.15 Recomendaciones de Contención / Respuesta

A modo de ejercicio de triage, y en el supuesto de que este comportamiento se observara en producción sin contexto de autorización previo, las acciones de respuesta recomendadas serían:

- Aislar el endpoint de la red mientras se confirma la legitimidad de la actividad.
- Recolectar y preservar los eventos Sysmon EID 1 asociados (ProcessGuid completos) para análisis forense.
- Decodificar y analizar cualquier comando PowerShell ofuscado antes de descartar o escalar la alerta.
- Validar la conexión de red saliente contra listas de reputación e inteligencia de amenazas, e identificar el proceso propietario del socket (PID) mediante herramientas del sistema operativo si la telemetría de red del SIEM no está disponible.
- Entrevistar al usuario/propietario del endpoint para confirmar si la actividad fue iniciada intencionalmente.
- Escalar a un analista de Nivel 2 si no puede confirmarse un origen legítimo dentro de un plazo razonable.

11. Limitaciones de Infraestructura

Durante la fase de implementación del laboratorio se presentaron dificultades técnicas propias del despliegue de un entorno Wazuh/OpenSearch en recursos limitados, las cuales requirieron trabajo de ajuste y resolución de problemas antes de alcanzar un entorno operativo estable.

Asimismo, si bien Sysmon Event ID 3 (conexión de red) fue habilitado y confirmado a nivel del sistema operativo Windows, dicho evento no fue ingerido exitosamente por Wazuh 4.12.0 en este entorno. Esta limitación no impidió completar la investigación del Incidente 001: Sysmon Event ID 1 y el resto de la telemetría de creación de procesos continuaron funcionando correctamente durante todo el proyecto, y

el analista compensa la ausencia de telemetría de red del SIEM mediante validación directa en el endpoint (netstat) y en el host remoto controlado (Netcat), según se documenta en la sección 10.10.

Estas limitaciones se documentan con fines de transparencia técnica y no constituyen el foco central de este informe.

12. Lecciones Aprendidas

- ProcessGuid y ParentProcessGuid son indispensables para una reconstrucción confiable de árboles de procesos; ProcessId/ParentProcessId no son suficientes por si solos, dado que el sistema operativo los reutiliza en el tiempo.
- Una alerta correctamente disparada no implica automáticamente un incidente de seguridad; el contexto (línea base, autorización, comportamiento esperado del entorno) es indispensable para la clasificación final.
- La ausencia de una fuente de telemetría específica (en este caso, Sysmon EID 3 en Wazuh) puede compensarse parcialmente mediante validación manual y correlacionada en los sistemas involucrados, aunque esto incrementa el esfuerzo de investigación.
- Es preferible documentar explícitamente lo que la evidencia no permite confirmar, antes que forzar una narrativa continua que no está respaldada por los datos disponibles.
- La ejecución de comandos ofuscados (-EncodedCommand) exige un paso adicional obligatorio de decodificación antes de cualquier evaluación de severidad.

13. Conclusión

El proyecto demostró la capacidad de operar el ciclo analítico completo de un SOC de Nivel 1 sobre un entorno de laboratorio real: generación de telemetría mediante Sysmon, detección mediante reglas locales de Wazuh, triage inicial, análisis detallado de procesos y líneas de comando, correlación basada en ProcessGuid, reconstrucción de línea de tiempo, mapeo a MITRE ATT&CK, evaluación de impacto y documentación de una conclusión defendible.

El valor central del proyecto no radica en la cantidad de alertas generadas ni en la complejidad de las técnicas ofensivas simuladas, sino en la disciplina analítica aplicada durante la investigación del Incidente 001: la distinción explícita entre evidencia confirmada, análisis, inferencia y evaluación; el reconocimiento honesto de una discontinuidad en la cadena de ProcessGuid en lugar de forzar la narrativa original; y la compensación metódica de una limitación real de telemetría mediante validación cruzada. Estas capacidades reflejan directamente el estándar de rigor esperado de un analista SOC Tier 1 / Junior Blue Team en un entorno profesional.